



Вариант №2622
Лабораторная работа №3
По дисциплине
Базы Данных

Выполнил:
Студент группы Р3112
Киселев Тимофей Васильевич

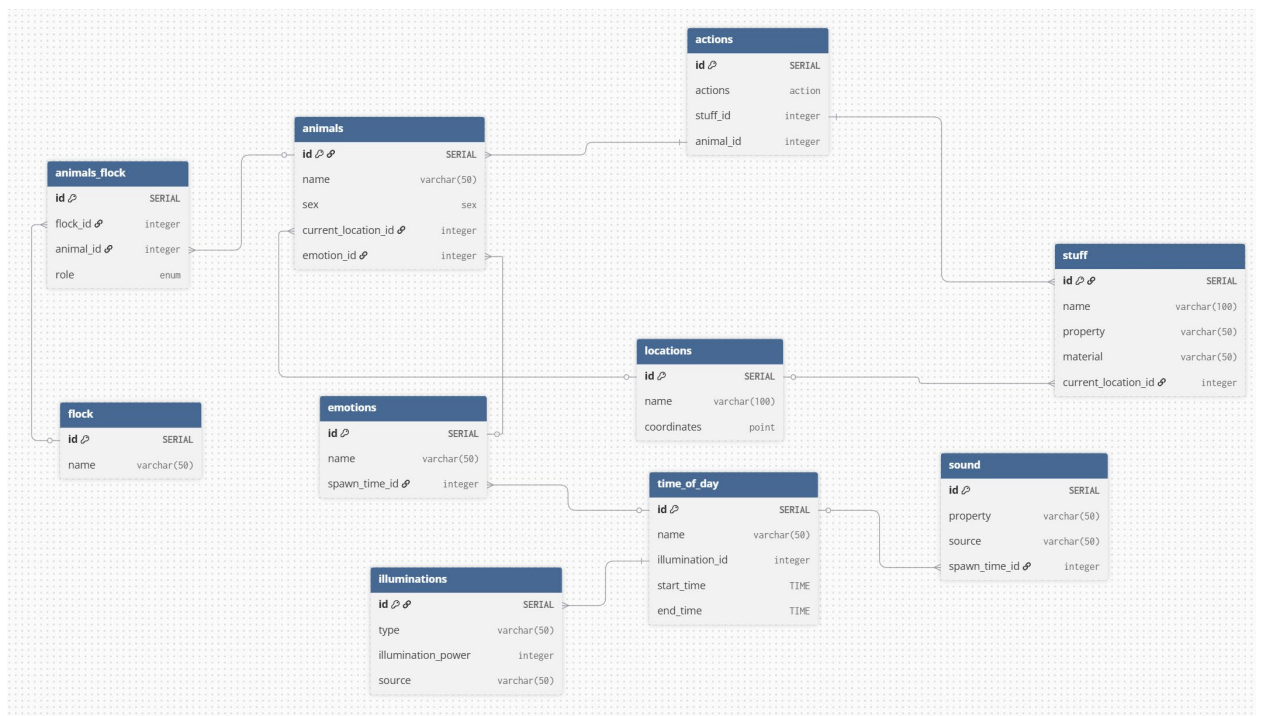
Преподаватель:
Вальц Мартин Эдуардович

1. Текст задания

Для отношений, полученных при построении предметной области из лабораторной работы №1, выполните следующие действия:

- Опишите функциональные зависимости для отношений полученной схемы (минимальное множество);
- Приведите отношения в 3NF (как минимум). Постройте схему на основе 3NF (как минимум).
- Опишите изменения в функциональных зависимостях, произошедшие после преобразования в 3NF (как минимум). Постройте схему на основе 3NF;
- Преобразуйте отношения в BCNF. Докажите, что полученные отношения представлены в BCNF. Если ваша схема находится уже в BCNF, докажите это;
- Какие денормализации будут полезны для вашей схемы? Приведите подробное описание.

Придумайте триггер и связанную с ним функцию, относящиеся к вашей предметной области, согласуйте их с преподавателем и реализуйте на языке PL/pgSQL.



2. Функциональные зависимости

animal: id – name, sex, current_location_id, emotion_id
animals_flock: id – flock_id, animal_id, role
flock: id – name;
actions: id – actions, stuff_id, animal_id
emotions: id – name, spawn_time_id
time_of_day: id – name, illumination_id, start_time, end_time
illuminations: id – type, illumination_power, source
sound: id – property, source, spawn_time_id
locations: id – name, coordinates
stuff: id – name, property, material, current_location_id

3. Нормальные формы

1NF: Отношение находится в 1NF, если все его атрибуты содержат только атомарные значения. Моя модель удовлетворяет 1NF, так как все атрибуты атомарны, и нет повторяющихся групп.

2NF: Отношение находится во 2NF, если оно находится в 1NF и все его неключевые атрибуты полностью функционально зависят от первичного ключа. Моя модель удовлетворяет 2NF, так как все неключевые атрибуты полностью функционально зависят от первичных ключей.

3NF: Отношение находится в 3NF, если оно находится во 2NF и не содержит транзитивных зависимостей. Моя модель удовлетворяет 3NF, так как все неключевые атрибуты зависят только от первичных ключей, и не содержат транзитивных зависимостей.

4. BCNF

Отношение находится в BCNF, если для каждой функциональной зависимости $X \rightarrow Y$, X является суперключом. Моя модель удовлетворяет BCNF, так как для всех функциональных зависимостей X является суперключом.

5. Денормализация

Объединение связанных таблиц: В некоторых случаях, объединение таблиц может уменьшить количество операций JOIN и ускорить обработку запросов. Например, можно рассмотреть объединение таблиц `animals` и `actions`, если часто запрашиваются данные о животном и его действии одновременно.

Добавление избыточных атрибутов: В некоторых случаях добавление избыточных атрибутов может улучшить производительность запросов. Например, если часто запрашивается количество животных в стае мы можем добавить атрибут `quantity` в `animals_flock`. Это позволит избежать операций подсчета при каждом запросе, однако необходимо будет обновлять этот атрибут при добавлении или удалении животных из стаи.

6. Функция на языке PL/pgSQL

Функция на языке PL/pgSQL для подсчета расстояния между двумя лабораториями.

```
CREATE OR REPLACE TRIGGER add_to_flock
  AFTER INSERT ON animals_flock
  FOR EACH ROW
  EXECUTE FUNCTION strongest_flock_after_add_to_flock();

DROP FUNCTION strongest_flock();

CREATE OR REPLACE FUNCTION strongest_flock_after_add_to_flock()
  RETURNS TRIGGER AS $$
```

```

DECLARE
    strongest_flock RECORD;
    flocks_rec RECORD;
    animal_ids int[];
    animal_id_1 int;
BEGIN
    SELECT * INTO strongest_flock FROM strongest_flock();
    RAISE NOTICE 'ID: %',strongest_flock.flock_id;
    RAISE NOTICE 'Name: %',strongest_flock.flock_name;
    RAISE NOTICE 'Leader id: %',strongest_flock.leader_id;
    RAISE NOTICE 'Animals power: %',strongest_flock.animals_power;
    FOR flocks_rec IN SELECT animals_flock.flock_id,role,animals.current_location_id,animals.id
FROM animals_flock
    JOIN animals ON animals_flock.animal_id = animals.id
    WHERE flock_id != strongest_flock.flock_id
    LOOP
        IF flocks_rec.role = 'Божак' and strongest_flock.leader_loc_id = flocks_rec.current_location_id
THEN
            SELECT array_agg(animal_id) INTO animal_ids
            FROM animals_flock
            WHERE flock_id = flocks_rec.flock_id;
            DELETE FROM animals_flock WHERE flock_id = flocks_rec.flock_id;
            DELETE FROM flock WHERE id = flocks_rec.flock_id;
            IF animal_ids IS NOT NULL AND array_length(animal_ids, 1) > 0 THEN
                FOREACH animal_id_1 IN ARRAY animal_ids
                LOOP
                    DELETE FROM animals WHERE id = animal_id_1;
                END LOOP;
            END IF;

        end if;
    end loop;
    RETURN new;
end;
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE FUNCTION strongest_flock()
RETURNS TABLE(
    flock_id int,
    flock_name varchar(100),
    leader_id int,
    animals_power int,
    leader_loc_id int

) as $$
DECLARE
    animals_in_flock int;
    flock_name varchar(100);
    flock_id int;
    scared_count int;
    leader_id int;
    temp_leader_id int;

```

```

    animal_record RECORD;
    flock_record RECORD;
    temp_leader_loc_id int;
    leader_loc_id int;
    animals_power int := 0;
    BEGIN
    FOR flock_record IN SELECT flock.id, flock.name FROM flock
        LOOP
            animals_in_flock:= 0;
            scared_count :=0;
            temp_leader_id := 0;
            FOR animal_record IN SELECT animals.current_location_id, animals.emotion_id,
animals_flock.animal_id, animals_flock.role FROM animals_flock
                JOIN animals ON animals_flock.animal_id = animals.id
                WHERE animals_flock.flock_id = flock_record.id
                LOOP
                    IF animal_record.emotion_id = 2 THEN
                        scared_count := scared_count +1;
                    end if;
                    animals_in_flock := animals_in_flock + 1;
                    IF animal_record.role ='Вожак' THEN
                        temp_leader_loc_id := animal_record.current_location_id;

                    end if;
                end loop;
            IF animals_in_flock - scared_count > animals_power THEN
                animals_power := animals_in_flock - scared_count;
                flock_name := flock_record.name;
                flock_id := flock_record.id;
                leader_id := temp_leader_id;
                leader_loc_id := temp_leader_loc_id;
            end if;
        end loop;
    RETURN QUERY SELECT flock_id,flock_name,leader_id, animals_power,leader_loc_id;
    RETURN;
end;

$$ LANGUAGE plpgsql;

```

7. Вывод

При выполнении лабораторной работы я познакомился с понятием нормализации и денормализации. Научился определять функциональные зависимости модели, а также анализировать последнюю на соответствие различным нормальным формам. Познакомился с процедурным языком PL/pgSQL. Изучил эффективные способы денормализации схемы базы данных и ситуации, в которых возможно их применение.